dnt API Manual

None

None

None

Table of Contents

1. dnt API Manual 3
   1.1 Install 3
   1.2 Packages 3
   1.3 Pipeline 3
   1.4 Units 3
2. Quickstart 4
   2.1 Detect and track a video 4
   2.2 Refine the tracks 4
   2.3 Re-classify, filter and label 4
   2.4 Working with the files directly 5
3. Changelog 6
   3.1 0.3.4.2 — 2026-11-05 6
   3.2 0.3.4 — 2026-10-01 9
   3.3 0.3.3 — 2026-09-26 9
4. API Reference 13
   4.1 API Reference 13
   4.2 dnt Package 14
   4.3 Detection 0
   4.4 Tracking 0
   4.5 Track Refinement 0
   4.6 Engine 0
   4.7 Filter 0
   4.8 Labeler 0
   4.9 Shared Utilities 0

1. dnt API Manual

Version 0.3.4.2 — see the changelog.

`dnt` (Detection and Tracking) turns roadside and intersection video into clean road-user tracks: it detects objects, tracks them across frames, refines and re-classifies the tracks, filters them by zones and lines, and draws the result back onto video. Every stage reads the previous stage's file and writes its own, so stages run on their own or chained.

1.1 Install

```
pip install dnt                # detection, tracking, filtering, labeling
pip install "dnt[refine]"      # track refinement: DINO/ReID encoders and VLM backends
pip install -e ".[dev]"        # from a checkout: tests and linting
pip install -e ".[docs]"       # from a checkout: build this manual
```

Python 3.11 or 3.12. Detection and deep ReID trackers use a GPU when one is available (`device="auto"` tries cuda, then xpu, mps and cpu). `ffmpeg` is needed by `Labeler` for the ffmpeg and `chrome_safe` methods. The `refine` extra splits into `refine-dino`, `refine-reid` and `refine-vlm` if you need only one encoder or backend.

1.2 Packages

Package	What it does	Main entry point
`dnt.detect`	Object detection with Ultralytics YOLO (v8, 11, 26) or RT-DETR; pedestrian-signal and timestamp readers	`Detector`
`dnt.track`	Multi-object tracking on BoxMOT; gap filling; track re-classification	`Tracker`, `ReClass`
`dnt.refine`	Splits reversals and ID switches, screens false tracks, merges duplicates, links fragments, fills gaps, with an audit ledger	`TrackRefiner`, `dnt-refine`
`dnt.filter`	Filters detections and tracks by zones and lines	`Filter`
`dnt.label`	Draws detections, tracks and shapes on video; cuts clips; exports frames	`Labeler`
`dnt.engine`	Box IoU, IoB, interpolation and gap clustering	`ious`, `iob`
`dnt.shared`	CSV readers and writers, class-name lists, downloads, multi-video synchronization	`Synchronizer`

1.3 Pipeline

```
video ─Detector─▶ detections ─Tracker─▶ tracks ─TrackRefiner─▶ refined tracks
                                                                    |
labeled video ◀─Labeler─ filtered tracks ◀─Filter─ re-classed ◀─ReClass
```

Files are headerless CSV text. Detections have 8 columns (`frame, res, x, y, w, h, conf, class`); tracks have 10 (`frame, track, x, y, w, h, score, cls, r3, r4`). Boxes are top-left `x, y` with width and height. The Quickstart runs the whole chain.

1.4 Units

- Box coordinates, widths and heights are in pixels of the source video frame.
- `Tracker` and `TrackRefiner` need the frame rate (from the video, or `--fps`) wherever they reason about time, such as how long a gap may be before a link is refused.

2. Quickstart

2.1 Detect and track a video

```
from dnt.detect import Detector, DetectorModel
from dnt.track import Tracker, BoTSORTConfig

det = Detector(model=DetectorModel.YOLO26x, conf=0.25, device="auto")
det.detect("cam1.mp4", iou_file="cam1_det.txt")        # detections, 8 columns

tracker = Tracker(config=BoTSORTConfig())              # the config picks the BoxMOT backend
tracks = tracker.track("cam1_det.txt", "cam1_track.txt", video_file="cam1.mp4")
```

Weights download on first use into `src/dnt/detect/yolo/models/`. A custom `weights=` path is relative to that directory unless it is absolute.

2.1.1 Choosing methods

- **Detector model**: pick a `DetectorModel` member (`YOLO26x` is the default; `n` and `s` sizes are faster). `classes=[...]` keeps only the classes you need.
- **Tracking backend**: pass one of `BoTSORTConfig`, `ByteTrackConfig`, `OCSORTConfig`, `StrongSORTConfig`, `DeepOCSORTConfig`, `HybridSORTConfig`, `BoostTrackConfig` or `SFSORTConfig`. Motion-only backends (ByteTrack, OC-SORT) need no ReID weights.
- **Several videos**: `Detector.detect_batch` and `Tracker.track_batch` take lists of files.
- **Saved settings**: `config.export_yaml(path)` writes a config and `Tracker(config_yaml=path)` reads it back.

2.1.2 Reading the output

Columns	Meaning
`frame`	Frame number in the source video
`track`	Track ID (detections have `res` instead, the detector's resolution field)
`x, y, w, h`	Box top-left corner, width and height, in pixels
`conf` / `score`	Detector confidence / tracker score
`class` / `cls`	Class index from the detector's class list
`r3, r4`	Reserved

2.2 Refine the tracks

```
from dnt.refine import RefineConfig, TrackRefiner

cfg = RefineConfig.defaults("vehicle")                 # or "person"
cfg.encoder.kind = "none"                              # motion only; no extra install
refiner = TrackRefiner(config=cfg)
refiner.refine("cam1_track.txt", "cam1_refined.txt", video_file="cam1.mp4")
```

The refiner splits tracks that reverse or switch IDs, drops false tracks, merges duplicates, links broken tracks and fills gaps. Every edit lands in a JSONL ledger next to the output. From a shell: `dnt-refine run cam1_track.txt --fps 10 --config refine.yaml --out clean.csv`. Appearance encoders and VLM verification are described under Track Refinement.

2.3 Re-classify, filter and label

```
from dnt.track import ReClass
from dnt.filter import Filter
from dnt.label import Labeler
```

```
# cars, buses and trucks; kind="ped" and kind="twowheel" are the other two
tracks = ReClass(kind="veh").re_classify("cam1_refined.txt", "cam1.mp4",
                                         out_file="cam1_veh.txt")

# keep the tracks that cross a zone (shapely polygons in pixel coordinates)
inside = Filter.filter_tracks_by_zones(tracks, zones=[zone_polygon])

Labeler().draw_tracks("cam1.mp4", "cam1_labeled.mp4", tracks=inside, tail_length=30)
```

- `Filter` also filters detections (`filter_iou` , `deduplicate_boxes`) and tracks by lines (`filter_tracks_by_lines`).
- `Labeler` also cuts clips (`clip` , `clip_by_time`) and exports frames (`export_frames` , `export_track_frames`).
- `VehReClass` and `PedReClass` can ask a vision-language model about hard cases when you give them a `vlm=` setting; see Re-classification.

2.4 Working with the files directly

The stage files are plain headerless CSV. `dnt.shared.files` holds the readers and writers, so you can load any stage into pandas without the stage classes, and `dnt.engine` has the box helpers (IoU, IoB, interpolation) used throughout. Browse the API Reference for every class and function.

3. Changelog

3.1 0.3.4.2 — 2026-11-05

3.1.1 Changed

- `dnt.refine` output for the vehicle target changes by default: tracks that reverse are split by the new `reversal` stage. Set `reversal: {enabled: false}` for the previous output.
- The person re-classifier is now `PedReClass`, with the arguments `ReClass` had. `ReClass` is a router: `ReClass(kind="ped" | "veh" | "twowheel", **kwargs)` builds `PedReClass`, `VehReClass` or `TwoWheelReClass` with `kwargs` and forwards `re_classify` and attribute reads to it. `ReClass(num_frames=...)` with keyword arguments still builds a `PedReClass`; positional arguments no longer do, because the first one is `kind`.

3.1.2 New

- Stage `reversal` in `dnt.refine` splits a vehicle track where its direction reverses (two vehicles that share one tracker ID). Link and dedup never join the two parts again, also through a third track or after a merge replaces one part's rows. See "Vehicle reversals" in the refine docs.
- `dedup` merges nested duplicate boxes: two tracks observed together that box one object (a torso inside a full body, or a bicycle and a motorcycle box on one machine) are a candidate even when densely co-observed (`dedup.nested_containment`, default 0.8). They need appearance, except when the boxes are all but identical (`dedup.duplicate_iou`, default 0.9), because boxes that cover each other occlude each other's crop.
- `TwoWheelReClass` decides whether two-wheeler tracks are bicycles (1) or motorcycles (3). It re-detects nothing: it matches each track against motorcycle and bicycle detections you already have (tables or headerless files) and the class the track carries. A clear majority decides; mixed evidence goes to an optional VLM; a larger overlap then the larger class vote decides otherwise. Each kept track gets one class on every row. Stand-up scooters and skateboards, and tracks the VLM calls pedestrian or scooter rider, are dropped, or with `scooter="keep"` kept as class 36. Per-track reasons are in `decisions`.
- `ReClass(rider_map=...)` sets the class each VLM rider answer stands for. Mapping an answer to `default_class` leaves the track a pedestrian, for pipelines where another stage owns bicycles and motorcycles.
- `ReClass` classifies riders. It now matches bicycle, motorcycle and skateboard/scooter detections (`match_class` defaults to `[1, 3, 36]`; it was `[1, 36]`) and can ask a VLM about tracks the overlap does not settle. A class that is the only one to overlap on at least `min_support` (default 0.5) of the sampled frames is accepted; a weak overlap, or several classes at once, goes to the model, which answers pedestrian, cyclist, motorcycle rider, scooter rider or unclear. A track with no overlap stays a pedestrian and is never sent. New arguments `vlm`, `vlm_crops`, `vlm_api_key`, `vlm_backend_factory` and `min_support`, and a `decisions` table. Without `vlm`, the best overlap still wins, so the output is unchanged apart from the added motorcycle class.
- `dnt.refine` merges interleaved duplicate tracks. A tracker sometimes alternates IDs on one person from frame to frame; the fill stage then interpolated each ID into the other's frames, so the output showed overlapping boxes on one person. The new `dedup` stage runs between screen and link: it finds pairs of tracks that move together but are almost never observed on the same frame, scores them (`MERGE` events), and merges the confident ones into the earlier ID, keeping the higher-score row where both have one. Pairs observed together often (two people walking side by side) are never merged. Uncertain merges go to the review page; merges that were accepted but blocked by a conflicting pair are shown there read-only. Set `dedup.enabled: false` for the old behavior. New config block `dedup`; the ledger header gains `absorbed` and the run summary gains `dedup`.
- `dnt.refine` looks at appearance. With a video and `encoder.kind` of `dino` (DINOv2, the default) or `reid` (torchreid OSNet), `refine` crops each box, skips crops that other boxes occlude, and embeds the rest. Stage 1 (ID-switch splits) and stage 3 (links) score with the embeddings, so a split or link that scores high enough is applied. The embeddings are cached next to the output as `OUT.features.npz`, and a rerun on the same inputs reuses them. New extras: `pip install 'dnt[refine-dino]'`, `'dnt[refine-reid]'`, or `'dnt[refine]'` for both. The first `dino` run downloads the model from the Hugging Face Hub (network needed; `transformers` caches it). `reid` finds torchreid's `FeatureExtractor` in the PyPI `torchreid` package and in deep-person-reid installed from GitHub (`pip install git+https://github.com/KaiyangZhou/deep-person-reid.git`), the alternative when the PyPI package does not provide it.

- VLM verification. With a video and `vlm.backend` set to `openai_compat` (any OpenAI-compatible server, such as vLLM or Ollama) or `anthropic`, `refine` builds an evidence image for each ID-switch split, link, false-track drop, or reclass in the uncertain band (and for each rider reclass without a subtype) and asks the model to choose an option. A confident answer decides the event (`VLM_ACCEPT` or `VLM_REJECT`) and can redirect the edit, for example a static drop into a reclass; a rider reclass that lacked a subtype is recorded as `AUTO_ACCEPT` with the subtype set; `unsure`, a low confidence (`vlm.min_conf`), a vote tie, or any error leaves it `HUMAN_PENDING`. `vlm.max_calls` is a hard limit on requests, retries included; answers are cached under `vlm.cache_dir`; `vlm.votes` asks a question several times. Questions are admitted to the budget closest-to-the-band-middle first within each stage (switch, screen, each link pass), earlier stages first; raise `max_calls` rather than expecting a global order. The run summary has a `vlm` entry with `calls`, `retries`, `cache_hits`, `failures`, `budget_skipped`, and `no_evidence` (events not asked because no evidence image could be made). Events left pending, with or without a backend, are collected on a static review page, `OUT.review.html` (signals only without a video), with their images in `OUT.review/`; **Export decisions** downloads `decisions.json`, which a later release will apply. Each entry is an object,
`{"accept": true|false, "new_cls": N (only when a class was picked),`
`"proposal_key": ..., "run_key": ...}`: event IDs are reused after a rerun, and `apply` will refuse or warn when a key does not match the ledger. `"accept"`, `"reject"`, and `{"accept": true, "new_cls": N}` stay valid in hand-written files. New extra: `pip install 'dnt[refine-vlm]'` (`openai` and `anthropic`; also part of `'dnt[refine]'`). Neither is a required dependency. See "Verification with a VLM" in the Track Refinement docs.
- Several VLM endpoints in one config: `vlm.endpoints` names them and `vlm.use` picks one (each may set `backend`, `base_url`, `model`, `api_key_env`, `api_key_file`, `max_tokens`, `extra_body`). `TrackRefiner` also reads `./.env` and the `.env` beside its config file, without overriding variables that are already set. `vlm.max_tokens` sets the output limit (default 1024 for `openai_compat`, which used to send a fixed 300); a reply cut off before its answer now names `max_tokens` in the error. `vlm.extra_body` passes provider switches (such as turning reasoning off) in every `openai_compat` request.
- VLM endpoint and key. `vlm.base_url` sets the endpoint of both backends (an OpenAI-compatible server, or a proxy or gateway in front of Anthropic); it must be an `http://` or `https://` URL with a valid host and port and without whitespace, a user name, password, query string, or fragment. The `anthropic` backend now honours `base_url` (it used to ignore it), so a `base_url` left in an older `backend: anthropic` config takes effect. A key sent over plain `http://` to a host other than this machine logs a warning. The key comes from `TrackRefiner(..., vlm_api_key=...)`, else from the file `vlm.api_key_file` names, else from the variable `vlm.api_key_env` names (`OPENAI_API_KEY` or `ANTHROPIC_API_KEY` by default); it must be one line of printable ASCII without whitespace (a key file's surrounding whitespace and byte order mark are stripped; a quoted key and a `NAME=value`, `NAME=` or `NAME==value` line are rejected), and it is never written to the config, the ledger, the summary, or a log line. This also applies to the default `OPENAI_API_KEY` and `ANTHROPIC_API_KEY` variables: a value with internal whitespace, a newline, quotes, or non-ASCII characters is now an error when the backend is built at the start of `refine` (a blank value counts as unset). `openai_compat` without `base_url` and without a key now fails before the run instead of calling api.openai.com with a placeholder. `dnt-refine run` takes `--vlm-backend`, `--vlm-model`, `--vlm-base-url`, `--vlm-api-key-env`, and `--vlm-api-key-file`, applied to the `--config` file before it is checked; switching the backend with `--vlm-backend` drops the file's `model`, `base_url`, and key settings (a file without a `backend` counts as `none`). There is deliberately no option for the key itself (shell history, process list).
- This release scores with motion only unless a video and an appearance encoder are given, and applies only the edits it is sure of: in-vehicle and duplicate false-track drops, rider reclasses whose subtype a ReClass hint settles, links across short gaps and static waits with a clear assignment margin, orphan drops, and filling. With an encoder, ID-switch splits and links are also scored by appearance and applied when they score high enough. Without a VLM backend (the default), other edits are capped below auto-accept, recorded as `HUMAN_PENDING`, and not applied yet: ID-switch splits found from motion alone, links across occlusions, links with an ambiguous assignment margin, and false-track drops of static objects or of mixed tracks. Rider reclasses whose subtype no ReClass hint settles are pending too, however high they score, unless a VLM backend names the subtype. With a VLM backend and a video, the VLM decides these edits when it is sure; the rest stay `HUMAN_PENDING` and go on a review page. In-vehicle drops need a context file with the vehicles' boxes (`context_file=`, or `--context`); without one the in-vehicle cue is skipped. Applying review decisions follows in a later release.
- `dnt.track.VehReClass` classifies vehicle tracks as car, bus or truck and writes the class into `cls` on every row. Each track goes through up to three layers: a size check (`hv_size`, at least `hv_min_count` real rows) that decides which re-detection votes are trusted; re-detection on 10 large frames spaced at least 5 frames apart, where the winning class needs 30% of the frames; and, when `vlm=` is set, one VLM question per undecided track with the options car, bus, truck and unclear, using `dnt.refine`'s backends and cache. Car means any light vehicle, vans and pickups included, and truck a heavy goods vehicle. COCO's truck also covers vans and pickups, so a bus or truck vote decides only on a size-flagged track; and a large box close to the camera is also what an SUV or pickup looks like, so a car vote decides only on a track that is not flagged. Where size and the vote disagree, the VLM is asked. A track still undecided stays car if re-detection leaned car, becomes truck if it is flagged heavy, and car otherwise. Set `hv_size` above the largest SUV or pickup the camera sees: with 240 on the Collins Ave clips, 13 SUVs,

pickups and a sedan were labelled truck, and 340 cleared them. `vr.decisions` shows which layer decided each track and, in `redetect_cls`, what re-detection voted.

- Stage 1 treats a reversal of heading as a motion break (new `switch.heading_reversal`, on for the vehicle target and off for the person target; `switch.heading_min_speed`, default 10 pixels per second, and `switch.ramps.heading`, default `[0.3, 0.8]`). A vehicle cannot turn around between two frames, but a few pixels per frame is below the Kalman filter's noise, so two vehicles joined at the edge of the frame (a box truck leaving, a van entering) scored 0.62 from appearance alone and were sent to the VLM, which judged the blurry edge crops the same vehicle. With the signal the split scores 0.97 and is accepted. On the 20-minute afternoon vehicle clip it changed one score and added one pending proposal, and on its pedestrians it changed nothing. The two new ledger signals are `heading` and `heading_cos`.
- Stage 3 refuses a normal-gate link across a reversed heading (new `link.normal_heading_check`, on for the vehicle target and off for the person target; the ledger records `heading`). The angle is between the mean velocity over the last `link.heading_seconds` (new, default 2.0) of one track and the first of the next, because a box clipped by the edge of the frame slows down just before it leaves. Position alone cannot tell an object leaving an edge from another entering there: on the afternoon clip a box truck and a van that the tracker had joined at the edge were split at frame 323 and then re-linked across a one-frame gap, so the split changed nothing. With the check the two stay apart. On that clip it removed 6 link events (1 applied, the wrong one; 5 pending proposals with a reversed heading), added none, and left the other applied links as they were. New `link.keep_splits` (off by default) also refuses to link the two halves of a track that stage 1 split. It is off because stage 3 had been healing two false auto-accepted splits on that clip.
- With a VLM backend and a video, the edits that stay capped below auto-accept without one (motion-only ID-switch splits, links across occlusions or with an ambiguous margin, static and mixed false-track drops, and rider reclasses without a subtype) are decided by the VLM when it is sure; the rest stay `HUMAN_PENDING`. With `vlm.backend: none` (the default), no decision changes, but `refine` now writes the review page (`OUT.review.html`) for the pending events, with their evidence images in `OUT.review/` when a video is given (the video is decoded for those images). `vlm.max_calls` applies to each `refine()` call, so under `refine_batch` it is a budget per video (each video gets its own runner; the answer cache is shared). The `refine` extra now also installs `openai` and `anthropic`; the new `refine-vlm` extra installs just those two.
- Config validation now rejects bad `vlm` settings: `votes` below 1, `min_conf` outside [0, 1], a negative `max_calls`, `max_concurrency` below 1, `timeout_s` or `vote_temperature` out of range, an empty `cache_dir`, and an `api_key_env` that is not an environment variable name (a pasted key; the message never repeats the value).
- The `refine-reid` and `refine` extras now also install `tensorboard`, which PyPI torchreid imports but does not declare.
- `refine` with a video now needs the encoder's package (`pip install 'dnt[refine-dino]'`) or `encoder.kind: none`. Before, it logged a warning and ran on motion alone. `dnt-refine run` exits with code 2 and names the extra when the package is missing. `TrackRefiner` accepts `encoder_factory=` to supply your own encoder.
- A context box now counts as a row's own detection, and is left out of the occlusion mask and of the link stage's occluders, when it overlaps a row of its frame at IoU 0.5 or more. Every such box counts, not only the best one per row, so a multi-class detection file of the same run (car, bus and truck proposals over one vehicle) no longer makes the vehicle look occluded by its own proposals. The cost: a real second object that overlaps a row at IoU 0.5 or more can no longer be told from a duplicate proposal, so it neither flags the row as occluded nor witnesses a gap. Before, it needed IoU 0.9 with any row, so a detection file of the same run as context flagged about a fifth of the rows as occluded. Stage 3 drops the rows' own detections from `context` with the same rule (`refine` does it before the stage; `score_candidates` called directly now expects `context` without them), so a detection file of the same run is supported as context: on three real pedestrian clips the share of rows flagged as occluded fell from about 20% to 7-11%.
- Stage 1 ignores the appearance of boxes whose longer side is below `switch.min_crop_px` (new, default 40 px), and stage 3 of those below `link.min_crop_px` (new, default 0: every clean crop); `0` turns a stage's filter off. A track whose boxes are all smaller gets no ID-switch proposal at all, and a link whose ends have no crop left is scored with appearance unknown. On three real 640x480 pedestrian clips, the two ID-switch splits applied on one clip came from small crops and each cut a single pedestrian (with the new default neither is proposed), while most links scored from small crops were correct. On the three clips (beginning, evening, middle), stage 1 proposed 2, 20 and 5 splits with the earlier default, 0, 3 and 1 with `switch.min_crop_px: 40`, and 42, 209 and 85 in motion-only mode; the filter removed 58-70% of the coarse samples (filtered counts are approximate: dense frames missing from the cached embeddings were left out). The feature cache key now includes the smaller of the two values and `FEATURES_VERSION` is 3, so caches of earlier runs are recomputed.
- Stage 3 applies more links: `link.accept_above` is 0.62 (was 0.80), and `link.ambiguous_cap` and `link.occluded_score_cap` are 0.60 (were 0.75). On three real pedestrian clips, nine of eleven pending links scoring 0.63-0.75 looked correct on review; with the new defaults eight of the eleven are applied. Of the other three, one has an ambiguous margin and stays pending, and two only

re-joined pedestrians cut by wrong ID-switch splits, which are no longer made. Links across occlusions and links with an ambiguous assignment margin still stay pending, because their caps stay below the threshold. The new threshold applies in motion-only mode too (`encoder.kind: none`, or no video), where the score is renormalized without appearance; on the three clips it applied one more link there. The score is multiplied by a border prior, `0.8 + 0.2 * b` with `b` 0 for a pair near the image border, so links near the border can now be applied too: 2 of the 8 applied links on the audited clips (beginning 98->134, middle 221->222) were border links, and both were among the audited, correct ones.

- `ReClass` reads every sampled frame once, in order, instead of reopening the video per track, and skips a frame the video cannot decode. With `verbose=True` its progress bar counts frames, not tracks. A video OpenCV cannot open now raises `ValueError`; before, every track silently got `default_class`. A summary warning is logged when sampled frames cannot be read.
- `dnt.track.ReClass` is always importable; the fallback that set it to `None` is removed.

3.2 0.3.4 — 2026-10-01

3.2.1 New

- `dnt.refine` track refinement: `TrackRefiner` (`refine`, `refine_batch`, used like `Tracker`) and `dnt-refine run`. They propose ID-switch splits, false-track drops and reclasses, and fragment links, drop orphans, and fill gaps. Every proposal is recorded in a JSONL ledger next to the output.
- This release scores with motion only and applies only the edits it is sure of: in-vehicle and duplicate false-track drops, rider reclasses whose subtype a ReClass hint settles, links across short gaps and static waits with a clear assignment margin, orphan drops, and filling. Other edits are capped below auto-accept, recorded as `HUMAN_PENDING`, and not applied yet: ID-switch splits found from motion alone, links across occlusions, links with an ambiguous assignment margin, and false-track drops of static objects or of mixed tracks. Rider reclasses whose subtype no ReClass hint settles are pending too, however high they score, because only a hint can choose the subtype in this release. In-vehicle drops need a context file with the vehicles' boxes (`context_file=`, or `--context`); without one the in-vehicle cue is skipped. Appearance encoders, VLM verification, review pages, and applying review decisions follow in later releases.

3.2.2 Changed

- `interpolate_tracks_rts` and `link_tracklets` moved to `dnt.refine`. `dnt.track.post_process` re-exports them, and `Filter.interpolate_tracks_rts` now calls `dnt.refine.interpolate`.
- `Tracker.track` finds each frame's detections with a sorted index instead of scanning the whole detection table every frame. The per-frame cost no longer grows with the length of the video (about 5 ms/frame on a 24 h file with 3M detection rows); track output is unchanged.
- `interpolate_tracks_rts` no longer uses rows flagged as filled (`interp` / `r3` equal to 1) as measurements, and accepts `protected_gaps`. Output for raw tracker files is unchanged.

3.3 0.3.3 — 2026-09-26

3.3.1 Important

- **Tracker settings now take effect.** In 0.3.2.x every tuning field of the tracker configs was silently ignored and BoxMOT's defaults were used. Settings now reach BoxMOT. Configs you never tuned keep their results: with the dependency versions in `tests/golden/reference-env.txt`, untuned tracks are unchanged from 0.3.2.4. Tuning fields now default to `None`, meaning "BoxMOT's value".
- Tracker configs are keyword-only: `BoTSORTConfig(ReIDWeights.X)` now raises `TypeError` (it previously set `model` by mistake). Use `BoTSORTConfig(reid_weights=ReIDWeights.X)`.
- Settings that cannot affect the chosen tracker raise `ValueError`: SF-SORT `det_thresh`, `max_age`, `min_hits`, `iou_threshold`, `asso_func`; BoostTrack `asso_func`. SF-SORT values outside the range BoxMOT would silently clamp also raise.
- `extra_kwargs`: 0.3.2.x factory keys that worked (`evolve_param_dict`, `tracker_config`, `per_class`, `reid_weights` on ReID trackers, and a `tracker_type` override on an untuned config) still work with identical results and are deprecated for 0.4. A `tracker_type`

override combined with the source config's own tuning fields (which 0.3.2.4 discarded) now raises. Keys that were silently ignored now raise; other tracker parameters are validated like fields.

- Saved configs: `from_dict()`, `import_yaml()` and `Tracker(config_yaml=...)` recognise 0.3.2.x output (YAML files and `to_dict()` dicts) and migrate it — untouched defaults follow BoxMOT (results unchanged), changed values take effect. Limitation: a value deliberately set equal to the old dnt default also migrates; the load warning lists every such field. 0.3.3 mappings carry `dnt_config_version: 2`. `XConfig(**old_dict)` is not migrated (dnt warns when it sees that pattern); use `from_dict`.
- `Synchronizer`: timestamps now continue across videos and `offsets` are applied (formula in the docstring); results change for every video after the first and wherever offsets are non-zero. A non-zero `offsets[0]` raises `ValueError`.
- Golden comparison: with the reference environment, untuned tracks are byte-identical to 0.3.2.4 for ByteTrack, BoT-SORT, StrongSORT, BoostTrack and SF-SORT and the `evolve_param_dict` passthrough. OC-SORT, Deep OC-SORT, HybridSORT and a `tracker_type` override to OC-SORT are exempt (reviewed defects BX16-NP2-*): they crash identically on 0.3.2.4 and 0.3.3 (see Known issues).
- `Tracker(device=...)` now uses the same device names as `Detector`: `auto`, `cpu`, `cuda`, `cuda:N`, `mps`, `xpu[:N]`. BoxMOT-style bare indices such as `device="0"` now raise `ValueError` (use `"cuda:0"`); `device="cuda"` now works. With `auto`, trackers use CUDA if available, then Apple MPS, else CPU (XPU hosts track on CPU, as before) — on Apple Silicon, ReID trackers now run on MPS instead of CPU.
- **`Detector` now defaults to the batched fast path.** It gains `fast` (default `True`) and `batch` (default `8`), read by `detect()` (and, through it, `detect_batch()`) on every call: with `self.fast` True, `detect()` redirects to `detect_fast(..., batch=self.batch)` instead of running its own per-frame pipeline — see `detect_fast()`'s changelog entry below for what that changes numerically (exact at `batch=1`; small, bounded, expected drift at `batch>1` with `half=True`, from FP16 batching numerics, not a defect). Existing code that constructs `Detector(...)` without `fast=`/`batch=` is now on this path by default for every `detect()`/`detect_batch()` call. Pass `Detector(fast=False, ...)`, or set `detector.fast = False` after construction, to keep the exact previous per-frame behavior for later calls (required for `show=True`, which `detect_fast()` doesn't support — calling `detect(show=True)` while `self.fast` is True raises `ValueError`). `fast`/`batch` are plain, mutable instance attributes, not `detect()`/`detect_batch()` call arguments — mirroring `conf`, `nms`, `device` and the rest of `Detector`'s existing settings.

3.3.2 Added

- `Detector` gains `imgsz`, `classes`, `rect`, `agnostic_nms` and `embed`, forwarded to Ultralytics `model.predict()` in both `detect()` and `detect_frames()`. They default to Ultralytics' own defaults (`640`, `None`, `False`, `False`, `None`), so existing calls are unaffected.
- `Detector` gains `class_names`, sugar for `classes`: resolves class names (e.g. `["car", "truck"]`) against `dnt.shared.util.load_class_dict()` (COCO names) and merges them into `classes`. Raises `ValueError` for an unknown name.
- `Tracker.track()`/`track_batch()` gain `verbose` (default `True`), matching `Detector.detect()`. `verbose=False` disables the per-video progress bar.
- `StrongSORTConfig` gains `min_conf`. BoxMOT's StrongSORT drops every detection scored below it before association, and the value it gets by default (from BoxMOT's YAML) is `0.6`, a hard cut with no low-score second stage. It could not be changed before; e.g. `StrongSORTConfig(min_conf=0.3)` now keeps weaker detections. The default (`None`) keeps `0.6`, so untuned results are unchanged.
- `Filter.deduplicate_boxes(detections, iou_thresh=0.45, containment_thresh=0.65)`: suppresses duplicate/nested-sub-box detections within each frame (e.g. a torso box nested inside a full-body box), keeping the higher-confidence box of each overlapping pair. Built on `dnt.engine.ious()`; both the IoU and containment checks it uses follow `cython_bbox.bbox_overlaps()`'s pixel-inclusive area convention (`(w+1)*(h+1)`), consistent with the rest of dnt's IoU-based matching.
- `Labeler.draw_tracks()` gains `message` (default `""`), matching `Detector.detect()`/`Tracker.track()`. It sets the progress-bar text shown after the video/batch position; `None` falls back to `input_video`, same as `Tracker.track()`'s fallback to the video file name. Ignored when `compress_message` is True.
- `Detector.detect_fast(input_video, ..., batch=None)`: same output as `detect()`, but batches `batch` (or `self.batch` if not given; see `Detector`) frames through the underlying Ultralytics `AutoBackend` in one forward pass instead of calling `model.predict()` once per frame. `detect()`'s per-frame path rebuilds an inference dataset and forces several CUDA synchronizations on every single-frame call, regardless of batch size; measured on an RTX 5070 Ti with RT-DETR-x, this is roughly 3-4x faster. At `batch=1` it reproduces `detect()` exactly (verified frame-for-frame, CPU and CUDA, YOLO and RT-DETR — see `detect_fast()`'s tests and docstring); at `batch>1` on CUDA with `half=True`, FP16's batched cuDNN/cuBLAS kernels can shift a small fraction of results

(measured: ~6% of frames, vs ~2.5% for `half=False`), which is inherent to batched half-precision GPU inference and not specific to this method. No `show=` preview (batching trades per-frame latency for throughput); use `detect()` for that.

- `Detector.detect()` and `detect_fast()` gain `return_df` (default `True`). With `return_df=False`, detections are only written to `iou_file` (then required) and `None` is returned. For a 24-hour video (~13–22 M detections) this avoids holding every row in memory.

3.3.3 Changed

- `detect_fast()` (and so `detect()` by default) now writes `iou_file` as it goes: each batch's rows are appended to `<iou_file>.part`, which is renamed to `iou_file` only once the whole range is done, so an existing `iou_file` is always complete. The finished file is byte-for-byte what the previous single write produced. An interrupted run leaves the `.part` file, and the next call with the same `iou_file` resumes from it (dropping and re-detecting its last frame, which may have been cut off); delete the `.part` file to start over. Rows are also kept as NumPy arrays rather than one dict per box (~64 vs ~431 bytes each), so memory stays far lower even with `return_df=True`.
- `detect_batch()` no longer builds the returned table it discards (it passes `return_df=False` when writing files), and with `is_overwrite=True` it deletes a video's stale `.part` file instead of resuming from it.
- `Tracker.track()` / `track_batch()` `message` now defaults to `""`, so the progress bar no longer shows the video file name (`Tracking 1 of 3` instead of `Tracking 1 of 3 - <file name>`). Pass `message=None` to get the file name back, or any string to show that instead.
- `Detector.detect()` / `detect_batch()` no longer print `Wrote detections to <file>` after writing a detection file. `detect_batch()` still returns the written file paths.
- `Labeler.draw_tracks()`'s progress bar no longer shows `input_video` by default outside of batch mode (`Generating labels` instead of `Generating labels <file>`), matching `Tracker.track()`'s `message=""` default. Pass `message=None` for the previous behavior. Its batch-mode check (`video_index` / `video_tot`) also now uses `is not None`, like `Tracker.track()`, instead of a truthy check, so `video_index=0` is now recognized as being in a batch.

3.3.4 Fixed

- `interpolate_tracks_rts(track_file=...)` no longer drops the first row (B3).
- `Filter.interpolate_tracks_rts` uses the package module instead of loading a second copy (B4).
- `Filter.filter_iou` works with zones (Polygon, MultiPolygon or list) (B5).
- `ReClass()` works with its defaults; `Detector(model=...)` accepts names such as `"rtdetr-x"` (B6).
- FP16 is used on indexed CUDA devices (`cuda:0`); the tracker only uses half precision on CUDA (B8).
- `detect_frames` rounds like `detect` (B9).
- `Labeler.draw_dets` accepts `Detector.detect()` output (B10).
- `SignalDetector` handles a short last batch (B13), no longer downloads ImageNet weights and loads on CPU-only machines.
- `Labeler` releases video captures; `export_track_frames(bbox=False)` writes frames (B14).
- `Tracker.track` no longer re-runs `update()` after a `TypeError`; empty detection files produce an empty track file.
- `track_batch` output names strip only a trailing `_iou`.
- `Detector` and `Segmentor` no longer pass `half=` to Ultralytics `model.predict()` on Ultralytics versions that deprecated it in favor of `quantize=` — that combination logged a `WARNING` on every single frame. `dnt._device.predict_precision_kwargs()` detects which the installed Ultralytics accepts and passes the matching kwarg; `Detector(half=...)` / `Segmentor(enable_half=...)` themselves are unchanged.
- `Filter.filter_iou()` and `Filter.deduplicate_boxes()` raised `KeyError` when passed the DataFrame `Detector.detect()` returns directly (named `Detector.DET_FIELDS` columns), since both were written for the positional/headerless layout (`pd.read_csv(file, header=None)`). Both now accept either layout and preserve whichever one the input had.

3.3.5 Packaging

- Requires Python 3.11+ (the code already did).

- `boxmot==16.0.11` pinned (0.3.2.4 allowed BoxMOT 25, whose API breaks tracking).
- The class-name files (`coco/openimages/voc.names`) now ship in the wheel (B7).
- The default ReID weight (`osnet_x1_0_msmt17.pt` , used by `Tracker()` 's default BoT-SORT) and the pedestrian-signal weight (`ped_signal.pt`) now ship in the wheel, so a clean install no longer downloads them on first use.
- Removed dependencies dnt never imports — install them yourself if you relied on them: torchaudio, opencv-contrib-python, faiss-cpu, faiss-gpu-cu12, cython, easydict, h5py, motmetrics, ninja, numpy_indexed, prettytable, pycocotools, scikit-image, shapelysmooth, sympy, tabulate, tensorboard, termcolor, thop, vidgear, openpyxl, matplotlib, gdown, lapx, loguru, yacs, scikit-learn.
- Top-level module names such as `detector` , `shared` and `filter` are no longer importable (they were side effects of `sys.path` manipulation, never public API).

3.3.6 Deprecated

- `StrongSORTConfig(max_dist=...)` : use `max_cos_dist` .
- `extra_kwargs` keys `evolve_param_dict` , `tracker_config` , `per_class` , `reid_weights` , `tracker_type` , `device` , `half` (removed in 0.4).

3.3.7 Known issues

- OC-SORT, Deep OC-SORT and HybridSORT (including `extra_kwargs["tracker_type"]` overrides targeting them) raise `TypeError: only 0-dimensional arrays can be converted to Python scalars` as soon as a lost track is re-detected. This is a BoxMOT 16.0.11 defect in `unfreeze()` under numpy >= 2 and was already present in 0.3.2.x. Use ByteTrack, BoT-SORT, StrongSORT, BoostTrack or SF-SORT until the BoxMOT upgrade planned for dnt 0.4. Building one of these trackers now emits a `UserWarning` , and the `TypeError` carries a note explaining the issue; the exception itself (type, message) is unchanged.
- `extra_kwargs["tracker_type"]` overrides from a non-ReID config (ByteTrack, OC-SORT, SF-SORT) to a ReID tracker now raise `ValueError` ; in 0.3.2.4 all five such combinations crashed inside BoxMOT.

4. API Reference

4.1 API Reference

The pages in this section are rendered from module docstrings and object signatures.

4.2 dnt Package

DNT package for detection and tracking.

This package provides detection and tracking functionality.